SPEECH SIGNAL ANALYSIS

Speech Signal Analysis and Processing

LAB WORK #1

Source-Filter Model · Formant Estimation · Prosody Synthesis
Vowel Space Modelling · GMM · BIC

Course Instructors
Prof. Eduardo Lleida · Prof. Antonio Miguel · Prof. Alfonso Ortega

Dept. of Electronics and Communications Engineering
University of Zaragoza · Academic Year 2025–2026

Contents

1 Introduction **2**
1.1 The Source-Filter Model of Speech Production . . . 2
1.2 The Source: Phonation and Fundamental Frequency . . . 2
1.3 The Filter: Vocal Tract Resonances and Formants . . . 3
1.4 The Engineering Perspective: Linear Predictive Coding . . . 3
1.5 The Cepstrum: Separating Source and Filter Analytically . . . 4
1.6 Fundamental Frequency Estimation: Praat's Autocorrelation Method . . . 5
1.7 Fundamental Frequency Estimation via the Cepstrum . . . 6
1.8 Formant Estimation: LPC and Cepstral Methods . . . 8
1.9 Statistical Modelling of the Vowel Space: GMM and BIC . . . 9
1.10 Connection to This Lab . . . 10

2 Workflow Overview **11**

3 Task 1: The Effect of Sampling Frequency **11**
3.1 Background . . . 11
3.2 Procedure . . . 12
3.3 Questions for the Report . . . 13

4 Task 2: Recording and Annotation in Praat **13**
4.1 The Recording . . . 13
4.2 Annotation Protocol . . . 14
4.3 Questions for the Report . . . 15

5 Task 3: Prosody and Glottal Synthesis **16**
5.1 Background . . . 16
5.2 Procedure . . . 16
5.3 Glottal Source Resynthesis . . . 17
5.4 Questions for the Report . . . 18

6 Task 4: Vowel Analysis in Python (Google Colab) **19**
6.1 Overview and Data . . . 19
6.2 Step 1 — Environment Setup . . . 19
6.3 Step 2 — Silence Removal and Segment Trimming . . . 19
6.4 Step 3 — Feature Extraction: F0, F1, F2 . . . 20
6.5 Step 4 — Gaussian Modelling of F1 . . . 21
6.6 Step 5 — F1/F2 Scatter Plot and Gaussian Mixture Model . . . 21
6.7 Questions for the Report . . . 22

7 Troubleshooting Guide **22**

8 Lab Report Requirements **23**

Lab Work #1: Speech Signal Analysis and Processing

Erasmus Mundus Master in Linguistic Data Science — Academic Year 2025–2026

1 Introduction

1.1 The Source-Filter Model of Speech Production

Speech is not a simple signal, it is the product of a sophisticated biological system that linguists describe in terms of *articulation* and that engineers model as a *signal processing pipeline*. The **Source-Filter Model**, originally formalized by Gunnar Fant (1960)[1], provides a unified framework that bridges both perspectives and underlies virtually all modern speech technology, from voice codecs to speech synthesis and automatic speech recognition.

The central insight is that the speech signal can be decomposed into two independent components that interact **multiplicatively in the frequency domain**, equivalently, **convolutionally in the time domain**:

$$S(f) = G(f) \cdot V(f) \cdot R(f) \tag{1}$$

where $G(f)$ is the **glottal source** spectrum, $V(f)$ is the **vocal tract transfer function** (the filter), and $R(f)$ accounts for the **radiation** effect at the lips. In the time domain, this multiplication becomes a convolution:

$$s(t) = g(t) * v(t) * r(t) \tag{2}$$

The key consequence of Equations 1–2 is **separability**: the phonetic content (encoded in V) and the prosodic content (encoded primarily in G) can, in principle, be extracted and manipulated independently. Figure 1 illustrates this three-stage pipeline together with the characteristic spectral shape of each component.

1.2 The Source: Phonation and Fundamental Frequency

The *source* originates in the **larynx**, where the **vocal folds** are set into quasi-periodic vibration by the subglottal airstream, a process called **phonation**. Each oscillation cycle releases a brief pulse of air into the vocal tract. The rate of these pulses is the **fundamental frequency** F_0, perceived as the *pitch* of the voice:

- Typical adult male range: $F_0 \approx 85$–$180\,\text{Hz}$
- Typical adult female range: $F_0 \approx 165$–$255\,\text{Hz}$

Spectrally, a periodic pulse train is a **harmonic series** with components at $F_0, 2F_0, 3F_0, \ldots$ The amplitude of these harmonics decreases at roughly $-12\,\text{dB/octave}$, giving the glottal source a characteristic falling spectral slope. It provides the raw material, but not the timbral quality, of the voice.

[1] Fant, G. (1960). *Acoustic Theory of Speech Production.* Mouton, The Hague.

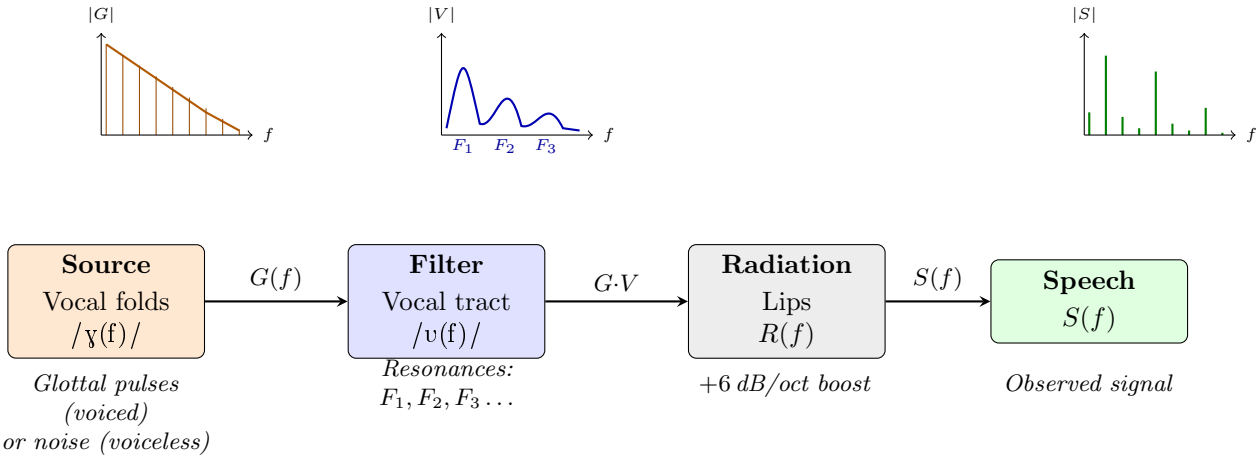

Figure 1: Block diagram of the Source-Filter Model. The glottal source $G(f)$ (falling harmonic spectrum for voiced sounds) is shaped by the vocal-tract filter $V(f)$ (resonances at formants F_1, F_2, F_3) and the lip-radiation term $R(f)$, producing the observed speech spectrum $S(f) = G(f) \cdot V(f) \cdot R(f)$.

Not all sounds require phonation. **Voiceless consonants** (/s/, /f/, /p/, /t/, /k/) use turbulent airflow or impulsive bursts instead of vocal-fold vibration. Their source approximates **bandlimited noise**, which is energy spread broadly across many frequencies rather than concentrated at harmonic peaks.

1.3 The Filter: Vocal Tract Resonances and Formants

The **vocal tract**, the air-filled tube extending from the glottis to the lips (approximately 17 cm in adults), acts as an **acoustic resonator**. Its transfer function $V(f)$ is shaped by the continuous repositioning of the articulators: the tongue body and tip, the jaw, the lips, and the velum. From a linguistics perspective these movements constitute *articulation*; from a signal processing perspective they tune the poles of a time-varying linear filter.

The resonant frequencies of the vocal tract are called **formants**, labelled $F_1, F_2, F_3, \ldots$ in ascending order of frequency. For vowels, the first two formants are sufficient to characterize phonemic identity:

- F_1 correlates with **tongue height** (jaw aperture): low vowels such as /a/ have *high* F_1; high vowels such as /i/ and /u/ have *low* F_1.
- F_2 correlates with **tongue backness**: front vowels /i, e/ have *high* F_2; back vowels /u, o/ have *low* F_2.
- Higher formants (F_3, F_4) contribute to voice quality and speaker identity but are less critical for vowel discrimination.

This direct correspondence between articulatory configuration and acoustic output is the model's great explanatory power: it tells us *why* a listener can identify the vowel /i/ regardless of whether the speaker is a child or an adult, whispering or shouting, the *filter shape*, not the source, carries the phonemic information.

1.4 The Engineering Perspective: Linear Predictive Coding

For students with a signal processing background, the source-filter model maps cleanly onto **all-pole autoregressive (AR) modelling**. If we approximate the vocal tract filter as a digital IIR filter of order p, its z-domain transfer function is:

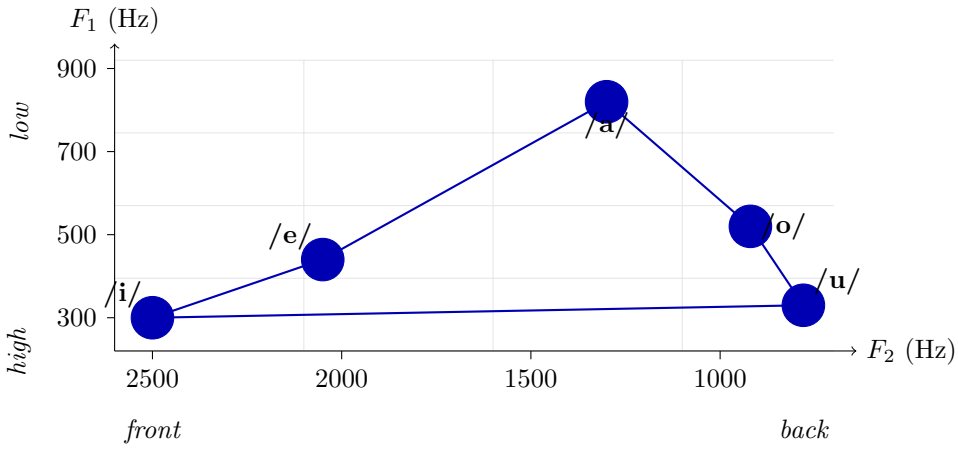

Figure 2: Vowel quadrilateral for the five cardinal vowels. F_2 (top axis, decreasing left-to-right) encodes front/back tongue position; F_1 (left axis, increasing downward) encodes tongue height. Approximate mean values for adult male speakers (Hz).

$$V(z) = \frac{G}{1 - \sum_{k=1}^{p} a_k \, z^{-k}} \tag{3}$$

Estimating the prediction coefficients $\{a_k\}$ from the speech waveform is the **Linear Predictive Coding (LPC)** algorithm, the backbone of voice compression (telephone codecs, MP3 speech layers) and a core frontend in classical automatic speech recognition. The complex poles of $V(z)$ occur in conjugate pairs whose frequency corresponds directly to formant frequencies.

The signal that remains after subtracting the LPC prediction from the original, the **LPC residual**, approximates the glottal source $g(t)$. This source-filter separation principle underlies voice conversion, text-to-speech synthesis, and the prosody manipulation you will perform in Task 2.

1.5 The Cepstrum: Separating Source and Filter Analytically

Recall that the source-filter model defines a *convolution* in the time domain (Equation 2). A powerful way to undo a convolution is to move into a domain where it becomes a *sum*, and that is precisely what the **cepstrum** achieves.

Definition

The **real cepstrum** of a signal $s(t)$ is defined by applying the Fourier transform twice, with a logarithm sandwiched in between:

$$c(q) = \mathcal{F}^{-1} \{\log |S(f)|\} \tag{4}$$

The resulting domain, replacing "frequency" with "**quefrency**" q, has units of *time*, but its interpretation is completely different: low quefrencies correspond to the **slowly-varying spectral envelope** (the vocal-tract filter), while high quefrencies correspond to the **fast harmonic ripple** imposed by the glottal source.

The deliberate wordplay (cepstrum ↔ spectrum, quefrency ↔ frequency, *liftering* ↔ filtering) was coined by Bogert, Healy & Tukey (1963)[2] and has remained part of the field's vocabulary ever since.

[2]Bogert, B. P., Healy, M. J. R., & Tukey, J. W. (1963). The quefrency alanysis of time series for echoes:

Why the logarithm matters

Under the source-filter model, in the log-spectral domain the convolution factorises into a sum:

$$\log |S(f)| = \log |G(f)| + \log |V(f)| + \log |R(f)| \tag{5}$$

Applying the inverse Fourier transform to Equation 5, each of the three terms maps to an additive component in the cepstrum, components that are **localised at different quefrency ranges** and can therefore be separated with a rectangular window called a **lifter**:

- **Low lifter** (keep low quefrencies): discards the harmonic structure and retains the spectral envelope $\log |V(f)|$, the phonetic content.
- **High lifter** (keep high quefrencies): discards the envelope and retains the harmonic ripple, whose *period* in quefrency equals $1/F_0$, the pitch information.

From cepstrum to MFCCs

In practice, the raw cepstrum is most often used through its variant, the **Mel-Frequency Cepstral Coefficients (MFCCs)**. The pipeline replaces the linear frequency axis with a **Mel filterbank** (warped to match the human auditory system's roughly logarithmic frequency resolution) before taking the logarithm and the Discrete Cosine Transform (DCT), the real-valued analogue of the IFFT in Equation 4. The first 12–13 coefficients capture the spectral envelope compactly, while appending their time derivatives (Δ and $\Delta\Delta$) adds dynamic information. MFCCs were the dominant speech feature in HMM-based ASR for three decades and remain a useful baseline today.

Connection to source-filter separation

The cepstrum thus operationalises source-filter separability. Computing the cepstrum of a voiced frame and *liftering* it gives you two separable signals from a single observed waveform:

Operation	**What you isolate**	**Linguistic correlate**
Low lifter $\rightarrow$ smooth spectrum	Vocal tract envelope $V(f)$	Phoneme / vowel quality
High lifter $\rightarrow$ harmonic peak	Fundamental period $1/F_0$	Pitch / intonation

This table summarises why the cepstrum is the natural mathematical language of the source-filter model, and why understanding it is prerequisite to reading any modern paper on speech synthesis, voice conversion, or speaker recognition.

1.6 Fundamental Frequency Estimation: Praat's Autocorrelation Method

Estimating F_0 from a speech waveform is non-trivial because the glottal source is only *quasi*-periodic, its amplitude varies over time, and voiced/unvoiced decisions must be made frame by frame. Praat implements the **Subharmonic-to-Harmonic Ratio (SHR) autocorrelation** algorithm described by Boersma (1993)[3], which is more robust to measurement errors than simple peak-picking in the autocorrelation function.

Cepstrum, pseudo-autocovariance, cross-cepstrum and saphe cracking. In M. Rosenblatt (Ed.), *Proceedings of the Symposium on Time Series Analysis*, pp. 209–243. Wiley, New York.

[3]Boersma, P. (1993). Accurate short-term analysis of the fundamental frequency and the harmonics-to-noise ratio of a sampled sound. *Proceedings of the Institute of Phonetic Sciences*, 17, 97–110. University of Amsterdam.

Short-time autocorrelation

The analysis proceeds on overlapping frames. For a Hann-windowed frame $x_w[n]$ of length N samples, the **normalised autocorrelation** is:

$$r(\tau) = \frac{\sum_n x_w[n]\, x_w[n+\tau]}{\sum_n x_w^2[n]}, \qquad \tau = 0, 1, \ldots, \tau_{\max} \tag{6}$$

For a voiced frame, $r(\tau)$ has a prominent peak at the lag $\hat{\tau}$ corresponding to the glottal period $T_0 = 1/F_0$, so:

$$\hat{F}_0 = \frac{f_s}{\hat{\tau}}, \qquad \hat{\tau} = \arg\max_{\tau \in [\tau_{\min},\, \tau_{\max}]} r(\tau) \tag{7}$$

The search range $[\tau_{\min}, \tau_{\max}]$ corresponds to the user-specified pitch range (`Minimum pitch` and `Maximum pitch` in Praat's `Sound: To Pitch (ac)...` dialogue). The default values are 75 Hz–600 Hz, appropriate for most adult voices.

Voiced/unvoiced decision

A frame is classified as **voiced** only when the autocorrelation peak strength exceeds a **voicing threshold** $r_{\min}$ (default 0.45 in Praat). Because the autocorrelation is normalised, $r(\hat{\tau}) = 1$ for a perfectly periodic signal and approaches 0 for white noise; values below $r_{\min}$ indicate insufficient periodicity and the frame is left unvoiced (no F_0 estimate is assigned).

Octave-error suppression

A classic failure mode is the *octave error*: the ACF also peaks at $2T_0$, $3T_0$, ..., and a naive picker may select the *subharmonic* lag $2\hat{\tau}$, halving the estimated pitch. Praat discourages such candidates via an **octave penalty** applied during Viterbi path-selection across frames, and the `Silence threshold` / `Voicing threshold` / `Octave cost` parameters in `To Pitch (ac)...` allow fine-grained control. The resulting F_0 contour is therefore smoother and less susceptible to octave jumps than raw per-frame peak-picking.

Path optimisation

Once per-frame candidates are computed, Praat selects the globally optimal F_0 trajectory using a **dynamic programming** (Viterbi) pass that penalises large inter-frame jumps. The total cost balances voicing probability with pitch-continuity, producing the smooth pitch curves visible in Praat's `Pitch` object and in the blue overlay of the SoundEditor.

1.7 Fundamental Frequency Estimation via the Cepstrum

The cepstrum provides an alternative route to F_0 estimation that does not rely on the autocorrelation function. The method follows directly from the high-lifter interpretation: for a voiced frame the harmonic structure of the spectrum creates a periodic ripple in $\log|S(f)|$, and that ripple manifests as a prominent peak in the cepstrum at the quefrency $q_0 = 1/F_0$.

Algorithm

Given a Hann-windowed frame $x_w[n]$ of length N samples at sampling rate f_s:

1. Compute the short-time DFT: $X(k) = \mathrm{DFT}\{x_w[n]\}$.

2. Form the log-magnitude spectrum: $L(k) = \log|X(k)|$ (adding a small floor ϵ to avoid $\log 0$).

3. Compute the real cepstrum via the IDFT:

$$c(q) = \mathrm{IDFT}\{L(k)\}, \qquad q = 0, 1, \ldots, N-1 \tag{8}$$

4. Restrict the search to the quefrency band $[q_{\min},\, q_{\max}]$ corresponding to the expected F_0 range $[F_{0,\min},\, F_{0,\max}]$:

$$q_{\min} = \left\lfloor \frac{f_s}{F_{0,\max}} \right\rfloor, \qquad q_{\max} = \left\lfloor \frac{f_s}{F_{0,\min}} \right\rfloor \tag{9}$$

   Typical values ($F_{0,\min} = 75\,\mathrm{Hz}$, $F_{0,\max} = 500\,\mathrm{Hz}$, $f_s = 16\,000\,\mathrm{Hz}$) give $q \in [32, 213]$ samples, i.e. quefrencies from 2 ms to 13.3 ms.

5. Pick the dominant peak:

$$\hat{q} = \arg\max_{q \in [q_{\min},\, q_{\max}]} c(q) \tag{10}$$

6. Convert to frequency:

$$\hat{F}_0 = \frac{f_s}{\hat{q}} \tag{11}$$

When multiple frames are analysed, the per-frame estimates are aggregated; reporting the **median** (rather than the mean) suppresses the effect of octave errors, which appear as isolated frames with $\hat{F}_0 \approx 2F_0$ or $\hat{F}_0 \approx F_0/2$.

Voiced/unvoiced decision

A voiced/unvoiced label can be assigned by comparing the cepstral peak amplitude $c(\hat{q})$ with a **threshold** θ: if $c(\hat{q}) < \theta$ the frame is deemed unvoiced. Unlike the normalised autocorrelation (which is bounded in $[-1, 1]$), the absolute scale of $c(q)$ depends on signal amplitude, so the threshold is usually set relative to the cepstral peak at $q = 0$:

$$\text{voiced} \Leftrightarrow \frac{c(\hat{q})}{c(0)} > \theta, \qquad \theta \approx 0.1\text{–}0.3 \tag{12}$$

Relation to the autocorrelation method

Both methods exploit the periodicity of a voiced signal; their key differences are:

Property	**Autocorrelation (Praat)**	**Cepstrum**
Signal domain	Time	Log-spectral / quefrency
Voiced/unvoiced	Normalised ACF peak $r(\hat{\tau})$	Relative cepstral peak
Octave robustness	Viterbi path with octave penalty	Median aggregation
Sensitivity to noise	Moderate	Higher (log amplifies low-SNR bins)
Computational cost	$O(N \log N)$ via FFT	$O(N \log N)$ (same order)
Post-smoothing	Viterbi across frames	Requires separate step

In practice, the autocorrelation method tends to be more accurate for clean speech, while cepstrum-based F0 tracking is useful when a source-filter decomposition is already being computed (e.g. for MFCC extraction) and a separate pitch estimate is needed at no extra FFT cost.

1.8 Formant Estimation: LPC and Cepstral Methods

Both LPC and the cepstrum provide routes to F_1/F_2 estimation, and both are rooted in the source-filter model: the goal is to recover the vocal-tract spectral envelope $V(f)$ and locate its resonance peaks.

Formant estimation via LPC

LPC fits an all-pole filter $\hat{V}(z)$ of order p whose frequency response tracks the spectral envelope of the current frame. The order is chosen as $p = f_s/1000 + 2$ (Praat's default), ensuring enough poles to model the expected number of formants in the bandwidth $[0, f_s/2]$ while preventing over-fitting.

The prediction coefficients $\{a_k\}_{k=1}^{p}$ are obtained by solving the **Yule-Walker equations** (equivalently, the Toeplitz autocorrelation system):

$$\begin{bmatrix} r(0) & r(1) & \cdots & r(p-1) \\ r(1) & r(0) & \cdots & r(p-2) \\ \vdots & & \ddots & \vdots \\ r(p-1) & r(p-2) & \cdots & r(0) \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_p \end{bmatrix} = \begin{bmatrix} r(1) \\ r(2) \\ \vdots \\ r(p) \end{bmatrix} \tag{13}$$

where $r(\tau)$ is the short-time autocorrelation of the pre-emphasised, windowed frame. The poles of $\hat{V}(z)$ are the roots of $1 - \sum_k a_k z^{-k} = 0$; a conjugate pair of roots $z_k = \rho_k e^{\pm j\omega_k}$ with ρ_k close to unity generates a sharp resonance at:

$$F_k = \frac{\omega_k}{2\pi} \cdot f_s = \frac{f_s}{2\pi} \angle z_k \tag{14}$$

The formants are extracted by sorting these frequencies in ascending order. In Praat, this is performed by `Sound: To Formant (burg)...`, which uses Burg's lattice implementation of the Levinson-Durbin recursion for numerical stability.

Formant estimation via the cepstrum

As derived in Section 1.5, applying a **low lifter** to the cepstrum retains only the slowly-varying spectral envelope. Transforming the liftered cepstrum back to the log-spectral domain yields a smoothed estimate $\widehat{\log |V(f)|}$ whose peaks are the formant candidates:

1. Compute the log-magnitude spectrum: $L(f) = \log |S(f)|$.
2. Apply the IFFT to obtain the cepstrum $c(q)$ (Equation 4).
3. Apply a low lifter (set $c(q) = 0$ for $q > q_c$, where $q_c \approx 2\,\text{ms}$ is a typical cut-off).
4. Apply the FFT to the liftered cepstrum to reconstruct the smoothed log spectrum $\hat{L}(f)$.
5. Find local maxima of $\hat{L}(f)$ in the ranges $[200, 900]\,\text{Hz}$, $[600, 2800]\,\text{Hz}$, etc. for F_1, F_2, ...

The cepstral method is faster to compute than polynomial rooting but less accurate for narrow, closely-spaced formants, because the lifter cutoff q_c controls a trade-off between envelope smoothness (too high q_c retains harmonic ripple) and frequency resolution (too low q_c over-smooths adjacent formants).

Comparison of the two approaches

Property	LPC	Cepstrum (low lifter)
Model assumption	All-pole AR(p)	Non-parametric smoothing
Key parameter	Order p	Lifter cutoff q_c
Closed formants	May merge into one pole	Harder to separate
Computational cost	$O(p^2)$ per frame	$O(N \log N)$ per frame
Robustness to noise	Sensitive to spectral tilt	More robust
Used in Praat	`To Formant (burg)`	Indirectly via MFCC pipeline

Both methods assume a **pre-emphasis** step (typically a first-difference filter $H(z) = 1-\alpha z^{-1}$, $\alpha \approx 0.97$) that compensates for the $-12\,\mathrm{dB/octave}$ glottal source slope and the $+6\,\mathrm{dB/octave}$ radiation effect, flattening the spectrum so that formant peaks are approximately equal in height before analysis.

1.9 Statistical Modelling of the Vowel Space: GMM and BIC

Because acoustic measurements such as F_1 and F_2 vary continuously across speakers, recording conditions, and speaking styles, a deterministic threshold is insufficient to classify vowels reliably. A principled alternative is to treat the formant observations as realisations of a **probability distribution** and to infer the distribution's parameters from data.

Univariate Gaussian model

When a single feature (e.g. F_1) is considered in isolation for a fixed phonemic category and gender group, the distribution is well approximated by a **univariate Gaussian** (normal distribution):

$$p(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \tag{15}$$

The parameters μ (mean) and σ^2 (variance) are estimated by **maximum likelihood**, which for a Gaussian reduces to the sample mean and sample variance. This model quantifies both the expected formant value and its natural variability across speakers, and it allows distributional overlap between neighbouring vowel categories to be expressed as a probability rather than a binary decision.

Gaussian Mixture Model

When the data comprise *multiple* phonemic categories simultaneously — as in the joint F_1/F_2 vowel space — a single Gaussian is too restrictive. A **Gaussian Mixture Model** (GMM)[4] represents the population density as a weighted sum of K multivariate Gaussians:

$$p(\mathbf{x}) = \sum_{k=1}^{K} \pi_k \, \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k), \qquad \sum_{k=1}^{K} \pi_k = 1, \quad \pi_k \geq 0 \tag{16}$$

where π_k is the **mixing weight** of component k, $\boldsymbol{\mu}_k \in \mathbb{R}^2$ is its centroid in the $[F_1, F_2]$ plane, and $\boldsymbol{\Sigma}_k$ is its 2×2 covariance matrix (encoding the spread and orientation of the ellipsoidal cluster). Each component naturally corresponds to one (vowel, gender) combination, so a linguistically

[4] Dempster, A. P., Laird, N. M., & Rubin, D. B. (1977). Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society: Series B*, 39(1), 1–38.

motivated choice for the vowel corpus used in Task 4 is $K = 6$ (three vowels $\times$ two genders). The parameters $\{\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}$ are estimated by the **Expectation-Maximisation (EM)** algorithm, which iterates between assigning soft-membership probabilities to each observation (E-step) and updating the Gaussian parameters to maximise the data log-likelihood (M-step)[5].

Bayesian Information Criterion for model selection

A practical question is: *how many components K should the mixture have?* More components always increase the data log-likelihood, but at the cost of model complexity and potential overfitting. The **Bayesian Information Criterion** (BIC)[6] penalises log-likelihood by a term proportional to the number of free parameters:

$$\mathrm{BIC}(K) = -2\,\ell(\hat{\boldsymbol{\theta}}_K) + d_K \ln N \tag{17}$$

where $\ell(\hat{\boldsymbol{\theta}}_K)$ is the maximised log-likelihood of the K-component model, N is the number of observations, and d_K is the number of free parameters (for full covariance matrices: $d_K = K(1+2+3) - 1 = 6K - 1$ in 2D). The preferred model is the one that **minimises** $\mathrm{BIC}(K)$. Because the penalty grows as $\ln N$, BIC is more conservative than the Akaike Information Criterion (AIC)[7] and tends to select smaller, more interpretable models, a desirable property when the ground-truth number of clusters has a linguistic interpretation.

1.10 Connection to This Lab

Every task in this session targets a specific component of the source-filter model:

- **Task 1 (Sampling Frequency):** Demonstrates how f_s limits the recoverable bandwidth, connecting Nyquist theory to perceptually salient contrasts such as the fricative distinction /s/ vs. /ʃ/.
- **Task 2 (Annotation):** Segmenting the waveform into phonemes corresponds to identifying regions with different source types (voiced periodic vs. voiceless noisy) and different filter configurations (different formant patterns).
- **Task 3 (Prosody Synthesis):** Extracting and resynthesizing the pitch contour isolates the *source* (F_0 trajectory) while discarding the filter — confirming that melody alone carries declarative/interrogative contrast.
- **Task 4 (Vowel Analysis in Python):** Automatic extraction of F_0, F_1, and F_2 from sustained vowel recordings using `Parselmouth`, followed by statistical modelling — Gaussian fitting of F_1 distributions and Gaussian Mixture Models on the joint F_1/F_2 vowel space.

Keep this decomposition in mind throughout the session. Every measurement you make is a window into one of these two separable layers of the speech signal.

Learning Goals

By the end of this session, you will be able to:

- **Capture and digitize audio:** Understand how a continuous acoustic pressure wave is converted into discrete digital samples and how f_s limits the recoverable bandwidth.

[5] Reynolds, D. A., & Rose, R. C. (1995). Robust text-independent speaker identification using Gaussian mixture speaker models. *IEEE Transactions on Speech and Audio Processing*, 3(1), 72–83.

[6] Schwarz, G. (1978). Estimating the dimension of a model. *The Annals of Statistics*, 6(2), 461–464.

[7] Akaike, H. (1974). A new look at the statistical model identification. *IEEE Transactions on Automatic Control*, 19(6), 716–723.

- **Annotate speech:** Place phonemic boundaries in Praat using acoustic cues from the waveform and spectrogram.
- **Synthesize prosody:** Generate a glottal pulse train that replicates the intonation pattern of a recorded utterance.
- **Preprocess audio programmatically:** Remove silences and trim transient regions using energy-based Voice Activity Detection in Python.
- **Extract acoustic features:** Use `Parselmouth` to compute F_0, F_1, and F_2 automatically, with gender-appropriate parameter settings.
- **Model acoustic distributions:** Fit univariate Gaussians to F_1 and Gaussian Mixture Models (GMM) to the F_1/F_2 vowel space; interpret the results linguistically.

2 Workflow Overview

- **Praat** (Tasks 1–3): Recording, manual phonemic segmentation, pitch visualisation, and resynthesis. Download: `https://www.fon.hum.uva.nl/praat/`
- **Python / Google Colab** (Task 4): All vowel analysis is performed programmatically. The notebook uses `Parselmouth` (Python bindings for Praat's analysis engine), `NumPy`, `SciPy`, `scikit-learn`, and `Matplotlib` for preprocessing, feature extraction, statistical modelling, and visualisation. No local installation is required, but the Colab environment provides all dependencies.

3 Task 1: The Effect of Sampling Frequency

3.1 Background

When a microphone converts a pressure wave into a digital file, the continuous signal is measured at discrete time instants at a rate called the **sampling frequency** f_s (in Hz or kHz). The **Nyquist–Shannon sampling theorem** states that all frequency components up to the **Nyquist frequency** $f_N = f_s/2$ are perfectly reconstructable; any energy above f_N is *aliased*, meaning that it folds back into the audible band and becomes distortion. This is why the choice of f_s directly determines the highest speech frequency that can be preserved:

f_s (kHz)	f_N (kHz)	Typical application
8	4.0	Telephone (PSTN), narrow-band ASR
11	5.5	AM radio quality
16	8.0	Wide-band telephony (HD Voice), modern ASR
22	11.0	Early multimedia / low-bitrate codecs
32	16.0	FM radio quality
48	24.0	Professional audio, video production

Speech energy relevant to phonemic contrasts extends from roughly **100 Hz** (F_0) to **8 kHz** (high fricatives such as /s/). Formants F_1/F_2 lie below 3 kHz, so they survive even narrow-band telephony; sibilant fricatives, however, require at least $f_s = 16$ kHz to be faithfully reproduced.

Target sentence:

"She sells seashells by the seashore."

/ʃiː sɛlz ˈsiːʃɛlz baɪ ðə ˈsiːʃɔːr/

This sentence contains four /s/ phonemes and three /ʃ/ phonemes, both are high-frequency fricatives whose spectral energy is concentrated above 4 kHz. They will be the first sounds to disappear as f_s decreases, making the degradation immediately perceptible even without prior phonetics training.

3.2 Procedure

You will record the declarative version of the target sentence (*"She sells seashells by the seashore."*) six times, one take at each sampling frequency listed below. Keep the microphone placement, vocal effort, and pace identical across all takes.

Step-by-step: Sampling Frequency Experiment

1. **Open the SoundRecorder.** In the Praat Objects window, go to `New` → `Record mono Sound...` The *SoundRecorder* window will open.

2. **Select your input device.** In the *Input source* drop-down, choose your microphone. Leave the sampling frequency at the default for now (you will change it in the next steps).

3. **Check the input level.** Click `Record` and speak at normal conversational volume. The level meter should reach roughly 70 % of full scale without entering the red (clipping) zone. Adjust microphone gain in your OS settings if needed, then click `Stop`. **Keep this gain setting unchanged for all six recordings.**

4. **Prepare the recording environment.** Move to a quiet room, maintain a constant mouth-to-microphone distance of 15–20 cm, and speak at the same pace and vocal effort for every take. These controls ensure that any perceived quality difference is due solely to f_s, not to recording conditions.

5. **Record at each sampling frequency.** Set the *Sampling frequency* drop-down to **8000 Hz**. Click `Record`, wait one second of silence, produce the target sentence once at a natural pace, wait one more second, then click `Stop`. Audition the take with `Play`; if there is clipping, background noise, or hesitation, redo it. Type the name `seashells_8k` in the *Name* field and click `Save to list & Close`. Repeat for each frequency:

Sampling frequency	Object name	File name
8 000 Hz	`seashells_8k`	`seashells_8k.wav`
11 025 Hz	`seashells_11k`	`seashells_11k.wav`
16 000 Hz	`seashells_16k`	`seashells_16k.wav`
22 050 Hz	`seashells_22k`	`seashells_22k.wav`
32 000 Hz	`seashells_32k`	`seashells_32k.wav`
48 000 Hz	`seashells_48k`	`seashells_48k.wav`

6. **Listen to each recording.** Select each Sound object and click `Play` (best with headphones). Note at which frequency the fricatives /s/ and /ʃ/ first sound muffled or almost disappear entirely.

7. **Inspect the spectrograms.** Select a Sound object and click `View & Edit`. Go to `Spectrogram` → `Spectrogram settings...` and set *Maximum frequency* to $f_s/2$ so the full bandwidth is visible. Compare the 8 kHz and the 48 kHz spectrograms side by side: the dark high-frequency striations of /s/ should be prominent in the 48 kHz version and absent in the 8 kHz one.

8. **Save all six files to disk.** Select each Sound object → `Save` → `Save as WAV file...` using the file names in the table above.

3.3 Questions for the Report

1. At which sampling frequency does the fricative /s/ in *"She"* first become unintelligible? Justify your answer by comparing the spectrograms side by side.

2. The vowel /iː/ in *She* has $F_1 \approx 280$ Hz and $F_2 \approx 2500$ Hz. Which of the six recordings still preserve F_2 accurately? Is F_2 already at risk at 8 kHz?

3. Compute the file sizes for a 3-second mono recording (16-bit PCM) at each f_s using the formula size $= f_s \times 2 \times t$ bytes. Plot size vs. f_s and discuss the quality/storage trade-off.

4. Why is 16 kHz the standard for most modern automatic speech recognition (ASR) systems? What would be lost by using 8 kHz instead?

4 Task 2: Recording and Annotation in Praat

4.1 The Recording

You will record the same sentence three times, each time with a different **sentence mode**. Keeping the segmental content identical across recordings isolates the prosodic dimension, changes in F_0 contour, duration, and intensity, as the *only* acoustic variable.

#	Mode	Utterance and expected F_0 pattern
1	Declarative	*"She left the door open."* Neutral falling contour: F_0 starts mid, peaks on *door*, then falls to low on *open*.
2	Interrogative	*"She left the door open?"* Rising contour: F_0 rises steeply on the final syllable *-pen*, ending above the start pitch.
3	Exclamative	*"She left the door open!"* High onset, sharp peak on *left*, then a steep fall; overall F_0 range much wider than the declarative.

Save each recording with a descriptive suffix: `she_left_decl.wav`, `she_left_inter.wav`, `she_left_excl.wav`. Use a quiet environment and maintain a constant distance from the microphone (15–20 cm) across all three takes. Follow these steps in Praat:

Step-by-step: Recording in Praat

1. **Open the SoundRecorder.** In the Praat Objects window, go to `New` → `Record mono Sound...` The *SoundRecorder* window will open.
2. **Select your input device.** In the *Input source* drop-down, choose your microphone. Set the *Sampling frequency* to **16000 Hz** (wideband quality).
3. **Check the input level.** Click `Record` and speak aloud to read the level meter. The bar should reach roughly 70 % of full scale without clipping (the red zone). Adjust microphone gain in your OS settings if needed, then click `Stop`.
4. **Record each version.** For each of the three modes (declarative, interrogative, exclamative): click `Record`, wait one second of silence, produce the sentence with the target intonation, wait one more second, then click `Stop`. Repeat within the same session so microphone placement stays constant.
5. **Audition the recording.** Click `Play` to review it. If there is background noise, clipping, or hesitations, click `Record` again to redo the take.
6. **Save to the Objects window.** After each take, type the corresponding name (`she_left_decl`, `she_left_inter`, `she_left_excl`) in the *Name* field and click `Save to list & Close`. You should end up with three Sound objects in the Objects window.
7. **Inspect the waveform.** Select the Sound object and click `View & Edit`. You should see a clear waveform with alternating voiced (regular oscillation) and voiceless (noise-like) regions. The spectrogram appears in the lower pane.
8. **Save to disk.** Select each Sound object in turn → `Save` → `Save as WAV file...` using the names `she_left_decl.wav`, `she_left_inter.wav`, `she_left_excl.wav`.

4.2 Annotation Protocol

Precise annotation is the bridge between raw acoustics and linguistic analysis. Before placing boundaries, familiarise yourself with both the orthographic form and the phonemic content of the target sentence. The symbols used below follow the International Phonetic Alphabet (IPA); a full interactive chart with audio examples is available at:

`IPA Interactive Chart (English) -- internationalphoneticassociation.org`

Orthography:	*"She left the door open."*	
Phonemic:	/ʃiː lɛft ðə dɔːr ˈoʊpən/	
Prosodic variants:	/ʃiː lɛft ðə dɔːr ˈoʊpən/	(declarative, falling F_0)
	/ʃiː lɛft ðə dɔːr ˈoʊpən/ ?	(interrogative, rising F_0)
	/ʃiː lɛft ðə dɔːr ˈoʊpən/ !	(exclamative, high peak + sharp fall)

Figure 3 shows a schematic TextGrid for the word *prism* (/ˈprɪzəm/), illustrating how the three annotation tiers relate to the waveform. Boundary lines (red) separate each segment; their placement on the actual spectrogram is guided by the acoustic cues described in the steps below.

Step-by-step: TextGrid Annotation in Praat

1. **Create the TextGrid.** Select the Sound object → `Annotate` → `To TextGrid...` Name the tiers: `word phoneme type`.

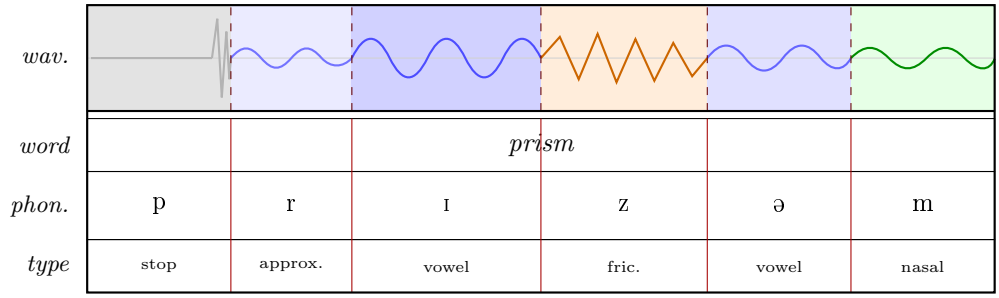

Figure 3: Schematic TextGrid for the word *prism* (/ˈprɪzəm/). The waveform (top) is shaded by phoneme type: grey = voiceless stop, blue = voiced periodic (sonorant/vowel), orange = fricative, green = nasal. Red boundary lines divide the three annotation tiers: `word`, `phoneme` (IPA), and `type`.

2. **Boundary placement.** Zoom in to the millisecond level. Apply the *zero-crossing* rule: place every boundary where the waveform crosses the centre line, to avoid clicks caused by discontinuities.

3. **Vowels.** Mark from the first to the last glottal pulse of the periodic region. Confirm visually with the dark horizontal formant bands in the spectrogram.

4. **Stops** (/p, t, k, b, d, g/). Separate the *closure* interval (flat line / silence) from the *burst* (vertical spike at release).

5. **Fricatives** (/s, f, ʃ/). Identify the onset and offset of the aperiodic (noise-like) energy band in the upper frequencies of the spectrogram.

6. **Label each interval.** Use IPA symbols for the `phoneme` tier, orthographic words for `word`, and the category (`vowel`, `stop`, `fricative`, `nasal`, `approximant`) for the `type` tier.

4.3 Questions for the Report

1. **Voicing transitions.** Identify two clear voicing boundaries in your annotation (e.g. the onset of /l/ and the closure of /t/ in *left*). Describe the acoustic cues (waveform regularity, spectral change) you used to place each boundary.

2. **Segmentation challenges.** Which phoneme boundary was hardest to locate precisely? Explain why, making reference to co-articulation or the acoustic properties of the sound in question.

3. **Tier design.** The TextGrid uses three tiers: *word*, *phoneme*, and *type* (voiced/voiceless/silence). Why is a three-tier design more useful for downstream analysis than a single phoneme tier alone? Give one concrete example of a corpus query that would be trivial with the *type* tier but cumbersome without it.

4. **Spelling vs. phonemes.** Count the total number of phonemes in your annotation of *"She left the door open"* and compare it with the number of letters in the orthographic form. For which words does the count differ most? Identify at least one instance of a silent letter and one instance where a single letter corresponds to two phonemes (or vice versa). What does this reveal about the relationship between English orthography and its phonological representation, and why does this matter for automatic grapheme-to-phoneme (G2P) conversion in NLP?

5 Task 3: Prosody and Glottal Synthesis

5.1 Background

Prosody refers to the suprasegmental properties of speech, including those acoustic dimensions that extend beyond individual phoneme segments and that speakers use to convey sentence type, focus, emphasis, and pragmatic meaning. The three canonical prosodic parameters are:

Prosodic parameter	Acoustic correlate	Perceptual correlate
Intonation	Fundamental frequency F_0	Pitch (melody)
Prominence / stress	F_0 peak + intensity + duration	Loudness, emphasis
Rhythm / tempo	Segment and syllable duration	Speech rate, weight

F_0 is determined by the **rate of vocal fold vibration**: faster vibration ⇒ higher pitch. It is therefore a property of the *source* in the source-filter model, entirely independent of the vocal tract shape. In connected speech, F_0 does not remain constant: it rises and falls continuously, tracing an **intonation contour** that listeners interpret as communicative intent. Compare the three realisations of the same sentence recorded in Task 2:

Mode	F_0 contour	Communicative function
Declarative	Falling ($H^*L\%$)	Assertion; speaker considers the proposition complete.
Interrogative	Rising ($L^*H\%$)	Yes/no question; speaker expects confirmation or denial.
Exclamative	High peak + steep fall ($\uparrow H^*L\%$)	Surprise or emphasis; wide F_0 range signals heightened affect.

Duration is the second prosodic cue: stressed syllables are *longer* than unstressed ones, and phrase-final syllables are lengthened (phrase-final lengthening). **Intensity** ($\propto$ amplitude^2) contributes to the percept of stress but is a weaker cue than F_0 or duration in English.

In this task you will isolate the F_0 contour from your three recordings, visualise it, and listen to a resynthesis that preserves only the prosodic layer — confirming that melody alone carries the declarative/interrogative/ exclamative distinction.

5.2 Procedure

You will repeat the following steps for each of the three recordings from Task 2 (`she_left_decl`, `she_left_inter`, `she_left_excl`).

Step-by-step: Pitch Visualisation and Prosodic Analysis in Praat

1. **Open the editor with the TextGrid.** In the Objects window, select *both* the Sound object (e.g. `she_left_decl`) and its corresponding TextGrid simultaneously (hold `Ctrl` and click both). Click `View & Edit`. The waveform, spectrogram, and annotation tiers will all appear in the same window, time-aligned.

2. **Enable the pitch track.** In the editor menu bar go to `Pitch` → `Show pitch`. A blue curve will overlay the spectrogram showing F_0 over time. Unvoiced regions (silence, fricatives) will have gaps in the curve.

3. **Adjust pitch settings if needed.** Go to `Pitch` → `Pitch settings...` Set the range to **75–300 Hz** for an average adult voice (lower bound for males ≈ 85 Hz; upper bound for females ≈ 255 Hz). If the curve looks erratic or has many spurious gaps, widen the range slightly until it tracks correctly.

4. **Inspect the contour against the annotation tiers.** Zoom into each word using `sel` (drag to select) or the `in/out` buttons. Check that the F_0 curve aligns with the voiced phoneme intervals in your `type` tier: the curve should be present over vowels and voiced consonants, and absent over silence and fricatives.

5. **Measure F_0 at key points.** Click at the temporal midpoint of the vowel /ɛ/ in *left*. The status bar at the top of the window shows the instantaneous F_0 value in Hz. Record the values for the declarative, interrogative, and exclamative versions in the table below.

	Declarative	**Interrogative**	**Exclamative**
F_0 at *left* (Hz)	...	...	...
F_0 at *open* (Hz)	...	...	...
Duration of /ɛ/ (ms)	...	...	...

6. **Take a screenshot of each contour.** Use `File` → `Draw visible sound and TextGrid...` (or a system screenshot) to capture the editor window showing the waveform, pitch curve, spectrogram, and annotation tiers simultaneously. Save one image per recording mode; you will include these in your report.

5.3 Glottal Source Resynthesis

In the source-filter model the glottal pulse train is the *source*: it carries all prosodic information (F_0 contour, rhythm) but none of the segmental identity of phonemes, which is encoded in the *filter* (vocal tract). Resynthesizing speech from the source alone lets you hear this separation directly.

Step-by-step: Glottal Source Extraction and Perceptual Analysis

1. **Extract the Pitch object.** In the Objects window, select `she_left_decl` → `Analyse periodicity` → `To Pitch...` (accept defaults). A `Pitch` object appears in the list.

2. **Synthesise the voiced excitation — three source types.** Using the original Sound and the `Pitch` object, generate three different source signals and listen to each one to compare their perceptual qualities:

   (a) **Pulse train.** Select the `Pitch` object → `To Sound (pulse train)...` (accept defaults). The result is a sequence of Dirac-like impulses spaced according to F_0. It carries the pitch contour but sounds maximally "robotic" because all harmonics have equal amplitude.

   (b) **Hum (band-limited pulse train).** Select the original Sound together with the `Pitch` object → `Sound` → `To Sound (hum)`. The result is spectrally smoother than the raw pulse train; the fundamental and lower harmonics are emphasised, giving a buzzing quality closer to a voiced laryngeal source.

   (c) **Sinusoidal (pure tone).** Select the `Pitch` object → `To Sound (sine)...`

(accept defaults). The result contains only the fundamental frequency F_0, with no harmonics at all. It sounds like a smoothly gliding whistle that perfectly traces the intonation contour.

Listen to all three and note how the perceived "naturalness" increases from (a) to (b), while (c) isolates the melodic contour most clearly of all.

3. **Extract the Intensity envelope.** Select the original Sound object → `Analyse periodicity` → `To Intensity...` (accept defaults). An `Intensity` object appears. Then select the `Intensity` object → `Down to IntensityTier`. An `IntensityTier` object appears.

4. **Modulate the amplitude.** Select the `IntensityTier` and the hum Sound object together → `Multiply`. A new Sound is created whose amplitude envelope follows the intensity of the original recording.

5. **Listen and describe.** Compare what you hear to the original recording. Answer:

   - Can you still tell that it is a question vs. a statement despite the robotic quality?
   - Which word carries the strongest pitch movement?
   - What is entirely absent from the resynthesis that was present in the original?

6. **Repeat for the interrogative and exclamative versions.** Rename the resulting Sound objects `pulse_decl`, `pulse_inter`, `pulse_excl` for clarity.

7. **Visual comparison.** Open `View & Edit` on one of the pulse trains. Observe that the waveform is a sequence of impulses whose spacing encodes F_0: closely-spaced impulses = higher pitch; wider spacing = lower pitch. Compare the inter-pulse intervals at the beginning and end of the interrogative version.

   *This is the first step towards the **Liljencrants-Fant (LF) glottal model**, where each pulse also has a physiologically-motivated asymmetric shape (slow opening, fast closure), giving even greater naturalness before the filter is applied.*

5.4 Questions for the Report

1. **Intonation contours.** Open the pitch track for each of the three recordings (`View & Edit` → `Pitch` → `Show pitch`). Sketch or include a screenshot of the F_0 contour for each version. At which word does the peak occur in the declarative? How do the rising and falling patterns differ between the interrogative and exclamative versions?

2. **Duration differences.** Using your TextGrid boundaries, measure the duration of the stressed vowel /ɛ/ in *left* across the three versions. Is it consistently longer in the exclamative? What does this tell you about the phonetic realisation of emphasis?

3. **Source-filter separation.** Compare the pulse-train resynthesis with the original recording. List three acoustic properties that are *preserved* in the resynthesis and three that are *lost*. Map each property to either the *source* or the *filter* component of the source-filter model.

4. **Sentence mode from prosody alone.** Play the three pulse trains (`pulse_decl`, `pulse_inter`, `pulse_excl`) to a classmate without revealing which is which. Can they identify the declarative, interrogative, and exclamative versions solely from the robotic signal? What does this tell you about the role of F_0 as a carrier of pragmatic meaning, independently of the segmental content?

5. **Inter-pulse intervals.** In the editor view of `pulse_inter`, identify the region corresponding to the final syllable *-pen*. Are the inter-pulse intervals getting shorter or longer towards the end? How does this relate to the rising F_0 contour of the interrogative? Express the relationship using the formula $T_0 = 1/F_0$.

6 Task 4: Vowel Analysis in Python (Google Colab)

6.1 Overview and Data

In this task you will analyse a corpus of sustained vowel recordings covering the three cardinal vowels /a/, /i/, /u/, produced by both male and female speakers. The full pipeline, including preprocessing, feature extraction, and statistical modelling, is implemented in Python using **Parselmouth**, which exposes Praat's analysis engine as a native Python library.

File naming convention

All audio files are 16-bit mono WAV at 16 kHz, named as:

`vowel_<vowel>_<gender><id>.wav`

where *vowel* ∈ {`a`, `i`, `u`}, *gender* ∈ {`m`, `f`}, and *id* is up to four-digit speaker index.

Filename	Content
`vowel_a_m1001.wav`	Vowel /a/ — male speaker 1001
`vowel_i_f1002.wav`	Vowel /i/ — female speaker 1002
`vowel_u_m1003.wav`	Vowel /u/ — male speaker 1003

Upload all files to a folder in your Google Drive and mount the drive at the beginning of the notebook.

6.2 Step 1 — Environment Setup

Install `praat-parselmouth` and import all required libraries (`numpy`, `pandas`, `matplotlib`, `scipy`, `scikit-learn`). Mount Google Drive and set the path to the audio folder in `AUDIO_DIR`.

Notebook — Step 1

Complete implementation in **Step 1** of `task4_vowel_analysis.ipynb`.

6.3 Step 2 — Silence Removal and Segment Trimming

Rationale

A sustained-vowel recording typically begins and ends with articulatory transients (onset coarticulation, phonation offset) that inflate variance and bias formant estimates. The preprocessing strategy is:

1. **Voice Activity Detection (VAD):** Mark a frame as voiced if its short-term intensity (in Praat's dB scale) satisfies

$$I(t) > I_{\text{FS}} + \theta_{\text{VAD}}, \qquad \theta_{\text{VAD}} = -40 \text{ dBFS}$$

where I_{FS} is the **theoretical full-scale intensity** for a 16-bit WAV file. Praat normalises 16-bit samples by dividing by $2^{15} = 32\,768$, so the maximum normalised amplitude is $A_{\max} = 1$. Using Praat's intensity reference $I_0 = 4 \times 10^{-10}\,\mathrm{W/m^2}$:

$$I_{\mathrm{FS}} = 10 \log_{10} \frac{A_{\max}^2}{I_0} = 10 \log_{10} \frac{1}{4 \cdot 10^{-10}} \approx 93.98 \text{ dB} \tag{18}$$

The effective threshold $I_{\mathrm{FS}} + \theta_{\mathrm{VAD}} \approx 53.98\,\mathrm{dB}$ is therefore **fixed and recording-level independent**: it depends only on the bit depth, not on how loud an individual speaker was recorded.

2. **Margin trimming:** Discard the first and last **250 ms** of the detected voiced interval. This 250 ms margin ensures that LPC formant tracks have fully stabilised (Hillenbrand et al., 1995)[8].

Notebook — Step 2

Complete implementation of `extract_steady_state()` in **Step 2** of `task4_vowel_analysis.ipynb`.

6.4 Step 3 — Feature Extraction: F0, F1, F2

F_0 via Praat autocorrelation

We extract per-frame F_0 and report the **median over voiced frames** (robust to octave errors). The pitch ceiling is gender-adapted: 300 Hz for males, 400 Hz for females.

Notebook — Step 3 (F0)

Implementation of `extract_f0()` in **Step 3** of `task4_vowel_analysis.ipynb`.

F_1/F_2 via LPC / Burg formant tracker

The critical parameter is `maximum_formant`: 5000 Hz for males, 5500 Hz for females (see Section 1.5 and Equation 14).

Notebook — Step 3 (F1/F2)

Implementation of `extract_formants()` in **Step 3** of `task4_vowel_analysis.ipynb`.

Batch processing all files

All WAV files are scanned with the regular expression `vowel_([aiu])_([mf])([0-9]+)[.]wav`, features are extracted for each token, and results are collected into a `pandas` DataFrame for subsequent analysis.

Notebook — Step 3 (batch loop)

Complete batch loop and summary statistics in **Step 3** of `task4_vowel_analysis.ipynb`.

[8] Hillenbrand, J., Getty, L. A., Clark, M. J., & Wheeler, K. (1995). Acoustic characteristics of American English vowels. *Journal of the Acoustical Society of America*, 97(5), 3099–3111.

6.5 Step 4 — Gaussian Modelling of F1

Statistical model

F_1 values for a given vowel and gender are approximately normally distributed across speakers (Hillenbrand et al., 1995)[9]. We model each (vowel, gender) group as a univariate Gaussian:

$$F_1 \mid v, g \sim \mathcal{N}\big(\mu_{v,g},\, \sigma^2_{v,g}\big) \tag{19}$$

Parameters $\mu_{v,g}$ and $\sigma_{v,g}$ are estimated by maximum likelihood (sample mean and standard deviation under a Gaussian model).

Notebook — Step 4

Six-panel histogram with fitted Gaussian PDF overlays in **Step 4** of `task4_vowel_analysis.ipynb`. The saved figure is `f1_gaussian_models.png`.

Expected pattern: $\mu_{/\mathrm{a}/} > \mu_{/\mathrm{i}/} \approx \mu_{/\mathrm{u}/}$ (tongue height), and $\mu_{\text{female}} > \mu_{\text{male}}$ for the same vowel (shorter vocal tract).

6.6 Step 5 — F1/F2 Scatter Plot and Gaussian Mixture Model

Scatter plot

Notebook — Step 5 (scatter)

F1/F2 scatter plot (axes inverted per phonetic convention) in **Step 5** of `task4_vowel_analysis.ipynb`. Saved as `f1f2_scatter.png`.

Gaussian Mixture Model

The six clusters in the F_1/F_2 plane (three vowels $\times$ two genders) are modelled as:

$$p(\mathbf{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}\big(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\big), \quad \mathbf{x} = [F_1,\, F_2]^\top, \quad K = 6 \tag{20}$$

Parameters are estimated by the EM algorithm (`scikit-learn`). Features are standardised first so that the larger F_2 range does not dominate.

Notebook — Step 5 (GMM)

GMM fitting and $1\sigma/2\sigma$ ellipse plot (back-transformed to Hz) in **Step 5** of `task4_vowel_analysis.ipynb`. Saved as `f1f2_gmm.png`.

Model selection via BIC

The linguistically motivated $K = 6$ should be validated against data-driven criteria. Compare BIC across $K = 2, \ldots, 10$:

Notebook — Step 5 (BIC)

BIC sweep $K = 2 \ldots 10$ with elbow plot in **Step 5** of `task4_vowel_analysis.ipynb`. Saved as `gmm_bic.png`.

[9] Hillenbrand, J., Getty, L. A., Clark, M. J., & Wheeler, K. (1995). Acoustic characteristics of American English vowels. *Journal of the Acoustical Society of America*, 97(5), 3099–3111.

6.7 Questions for the Report

1. **Preprocessing effect.** Re-run the feature extraction without the 250 ms margin (set `margin_ms=0`). Compare median F_1 and F_2 before and after trimming for each vowel and gender. Which vowel shows the largest shift? Why?

2. **Gender differences in F_0.** Report $\bar{F}_0 \pm \sigma_{F_0}$ for male and female speakers. Is the ratio $\bar{F}_{0,\text{female}}/\bar{F}_{0,\text{male}}$ consistent with the expected ≈ 1.7 (based on vocal-fold mass differences)?

3. **Gaussian fit of F_1.** For each (vowel, gender) group, report $\hat{\mu}$ and $\hat{\sigma}$ from Equation 19. Which vowel shows the largest within-group variance? Suggest a phonetic explanation (coarticulation, vowel-inherent spectral change).

4. **Distributional overlap.** Compute the **Bhattacharyya coefficient** between the /a/ and /u/ F_1 distributions for each gender. Let $\bar{\sigma}^2 = (\sigma_1^2 + \sigma_2^2)/2$; then:

$$\text{BC} = \exp\left(-\frac{(\mu_1 - \mu_2)^2}{8\,\bar{\sigma}^2} - \frac{1}{2}\ln\frac{\bar{\sigma}^2}{\sigma_1\sigma_2}\right)$$

   BC ≈ 1 means complete overlap; ≈ 0 means full separability. How well does F_1 alone distinguish /a/ from /u/?

5. **GMM covariance structure.** Inspect $\{\mathbf{\Sigma}_k\}$. For which vowel/gender cluster is the F_1–F_2 correlation strongest (most tilted ellipse)? Relate this to articulatory coupling between tongue height (F_1) and tongue backness (F_2).

6. **BIC model selection.** Does the BIC curve have a clear minimum? If the optimal K differs from 6, what factors (dialectal variation, speaking style) could explain the discrepancy? Would you trust a purely data-driven K for a vowel classifier?

7. **Vowel triangle comparison.** Plot the GMM centroids on the vowel quadrilateral alongside the reference values in Figure 2. Is your triangle expanded or compressed? How might microphone type or room acoustics systematically bias the formant estimates?

7 Troubleshooting Guide

- **Parselmouth import error:** Ensure you ran `!pip install praat-parselmouth` *before* the import cell, and restart the Colab runtime if the package was just installed.

- **Voiced segment too short (None returned):** Raise `vad_dbfs` to -50 dBFS (less restrictive threshold), or reduce `margin_ms` to 150 ms for short recordings.

- **Erratic F_0:** Raise the pitch ceiling (`ceil` in `extract_f0`): try 350 Hz for males, 500 Hz for high-pitched females.

- **Formant outliers:** Tighten the sanity bounds in `extract_formants` (e.g. $F_1 \in [200, 1000]$ Hz), or increase the window length to 30 ms for low-pitched voices.

- **GMM not converging:** Increase `n_init` to 20, or start with `covariance_type='diag'` before switching to `'full'`.

- **Erratic Praat pitch (Tasks 2–3):** Default range is 75–600 Hz. Try 50–300 Hz for deep voices or 100–500 Hz for high voices.

- **Misaligned formants (Tasks 2–3):** Set *Maximum Formant* to 5000 Hz for adult males and 5500 Hz for females in `Formant settings`.

8 Lab Report Requirements

Submit your completed `.ipynb` notebook (with all cell outputs visible) together with a PDF report containing:

1. **Tasks 1–3 (Praat):** One representative spectrogram per sampling frequency (Task 1); one TextGrid annotation screenshot (Task 2); three pitch-contour overlays for declarative, interrogative, and exclamative (Task 3).

2. **Task 4 — Preprocessing:** A table comparing median F_1 and F_2 with and without the 250 ms margin for each (vowel, gender) group.

3. **Task 4 — F_0 summary:** $\bar{F}_0 \pm \sigma_{F_0}$ per (vowel, gender) group, with a brief comment on gender differences.

4. **Task 4 — Gaussian F_1 figure:** The three-panel figure from Step 4, with $\hat{\mu}$ and $\hat{\sigma}$ annotated on each panel.

5. **Task 4 — GMM figure:** The F_1/F_2 scatter plot with GMM ellipses and the BIC curve. Annotate centroid positions with vowel labels.

6. **Written analysis:** Answers to all seven questions in Section 6.7, supported by the figures and numerical results from the notebook.